

HIVE CONSULT

Digital Forensics Examination Report

Project Chimera | Case HIVE-FSA-009E

Case reference	HIVE-FSA-009E
Project	Project Chimera
Organisation	HIVE CONSULT
Exhibit	HIVE-FSA.pdf (confidential board audit document)
Examiner	Philip Oppong Adanse
Report date	4 October 2026
Classification	Confidential
Outcome	4 genuine flags recovered. 2 decoy flags identified and set aside.

1. Summary

I was asked to examine a confidential board audit PDF recovered from a workstation where client evidence was suspected to have been moved or changed. The document opens as a normal one-page report. The case brief warned that it could hold more evidence than the PDF viewer shows, so I did not stop at what the file displays.

I found that the PDF carried a hidden ZIP archive after its normal end marker. Inside the archive was a long block of base64 text, and that text decoded into an email. The email had an attachment that was named like a data file but was really a JPEG image. That image also had a second ZIP archive attached after its own end marker, and inside the second archive was a PDF holding the last flag.

Along the way I found six flag-style strings. Four are genuine and two are decoys. Section 6 lists each one and explains how I decided.

2. Case Details and Scope

Scenario	HIVE CONSULT suspects manipulation and unauthorized movement of confidential client evidence. A board audit document was recovered from the affected workstation. It opens normally but may have more evidence associated with it.
Objective	Recover four genuine flags and tell real evidence apart from misleading or decoy flags.
Difficulty	Advanced
Scope	The single exhibit HIVE-FSA.pdf and anything recovered from it.
Approach	I worked on a copy in a separate folder and recovered each layer by hand, checking the real file type at every step.

3. Tools and Environment

Item	Detail
Analysis system	Kali Linux 2026.1 (amd64), running in VMware Workstation
User account	cobra6golf
Working folder	~/chimera, with s1 for the first archive and s1/s2 for the second
File identification	file, od, strings
Hashing and metadata	sha256sum, exiftool (version 13.50), pdftinfo
Searching and carving	grep, tail, dd, Python 3
Archives and encoding	unzip, base64, Python email module
Viewing	xdg-open (image), pdftotext (PDF text)

Times in the screenshots come from the Kali system clock, which was set to UTC-4 (EDT).

4. Exhibit and Integrity Record

I copied the PDF from the Desktop into ~/chimera and hashed the copy before doing anything else (Figure 1).

File name	HIVE-FSA.pdf
SHA-256	b2590294b85a82595e02ccacf4826b41a509736040d5520b8830bb7266415209
Size	192,947 bytes (about 193 kB)
File type	PDF document, version 1.4, 1 page
Working copy created	3 October 2026, 20:31 (EDT)
Permissions	-rw-----

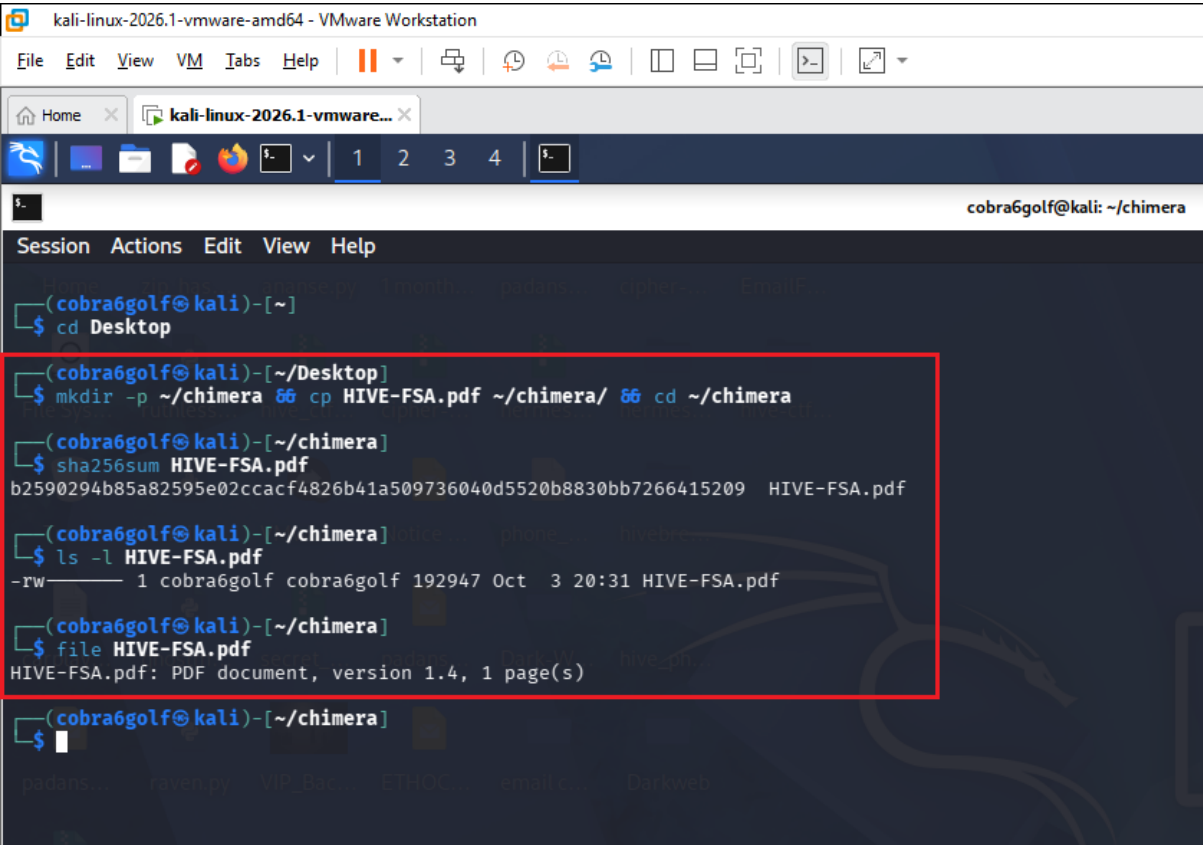


Figure 1. Working copy of HIVE-FSA.pdf created in ~/chimera, then hashed and identified (sha256sum, ls, file).

Box	What it shows	Why it matters
Red	Copy of the PDF into ~/chimera, then sha256sum, ls -l and file.	This creates the working copy and records its SHA-256, size (192,947 bytes) and type (PDF 1.4, 1 page) before any analysis.

The visible content is short. It is a board security audit summary for Project Chimera marked Confidential. It covers four observations that is the limited access to restricted evidence repositories, quarterly review of administrative access, monitoring of removable media, and hashing with chain-of-custody controls. Nothing in the visible text points to hidden material.

5. Examination Steps

Each screenshot sits under the step it supports. In the screenshots, the red boxes mark the commands and output I relied on, and green boxes mark the key findings. The table under each screenshot explains what each box shows and why it matters prior to this investigation.

5.1 Checking the PDF structure

I ran ExifTool on the PDF (Figure 2). It reported PDF version 1.4, not linearized, and one warning **Invalid xref table**. I did not treat the warning as proof of anything, but it told me the file structure was not clean.

```

kali-linux-2026.1-vmware-amd64 - VMware Workstation
File Edit View VM Tabs Help
kali-linux-2026.1-vmware... x
cobra6golf@kali:
Session Actions Edit View Help
(cobra6golf@kali)-[~/chimera]
$ exiftool HIVE-FSA.pdf
ExifTool Version Number      : 13.50
File Name                    : HIVE-FSA.pdf
Directory                    : .
File Size                    : 193 kB
File Modification Date/Time   : 2026:10:03 20:31:38-04:00
File Access Date/Time        : 2026:10:03 20:31:49-04:00
File Inode Change Date/Time   : 2026:10:03 20:31:38-04:00
File Permissions              : -rw-----
File Type                    : PDF
File Type Extension          : pdf
MIME Type                    : application/pdf
PDF Version                  : 1.4
Linearized                   : No
Warning                      : Invalid xref table
(cobra6golf@kali)-[~/chimera]
$

```

Figure 2. ExifTool output for HIVE-FSA.pdf, including the "Invalid xref table" warning.

Box	What it shows	Why it matters
Red	ExifTool fields: size 193 kB, PDF 1.4, Linearized: No, Warning: Invalid xref table.	This confirmed the basic details and shows the file structure was not clean. The size is far larger than one short page needs.

The file size was the stronger clue. The visible page holds only a few short paragraphs, yet the file is 192,947 bytes. A one paged pdf file like this should normally have a size of 2 to 5KB. A file like that should be much smaller, so I looked for data that the viewer does not use.

5.2 Finding data after the PDF end marker

A PDF normally finishes with the marker `%%EOF`. I searched for it with `grep -abo`, and I also searched for a ZIP file signature `PK\x03\x04` (Figure 3).

Search	Byte offset	What it means
<code>%%EOF</code>	2504	End of the real PDF. Only one marker was present.
<code>PK\x03\x04</code>	2510	Start of a ZIP archive, 6 bytes after the PDF ended.
<code>PK\x03\x04</code>	2634	Second ZIP entry header.
<code>PK\x03\x04</code>	2707	Third ZIP entry header.

Search	Byte offset	What it means
PK\x03\x04	2815	Fourth ZIP entry header.

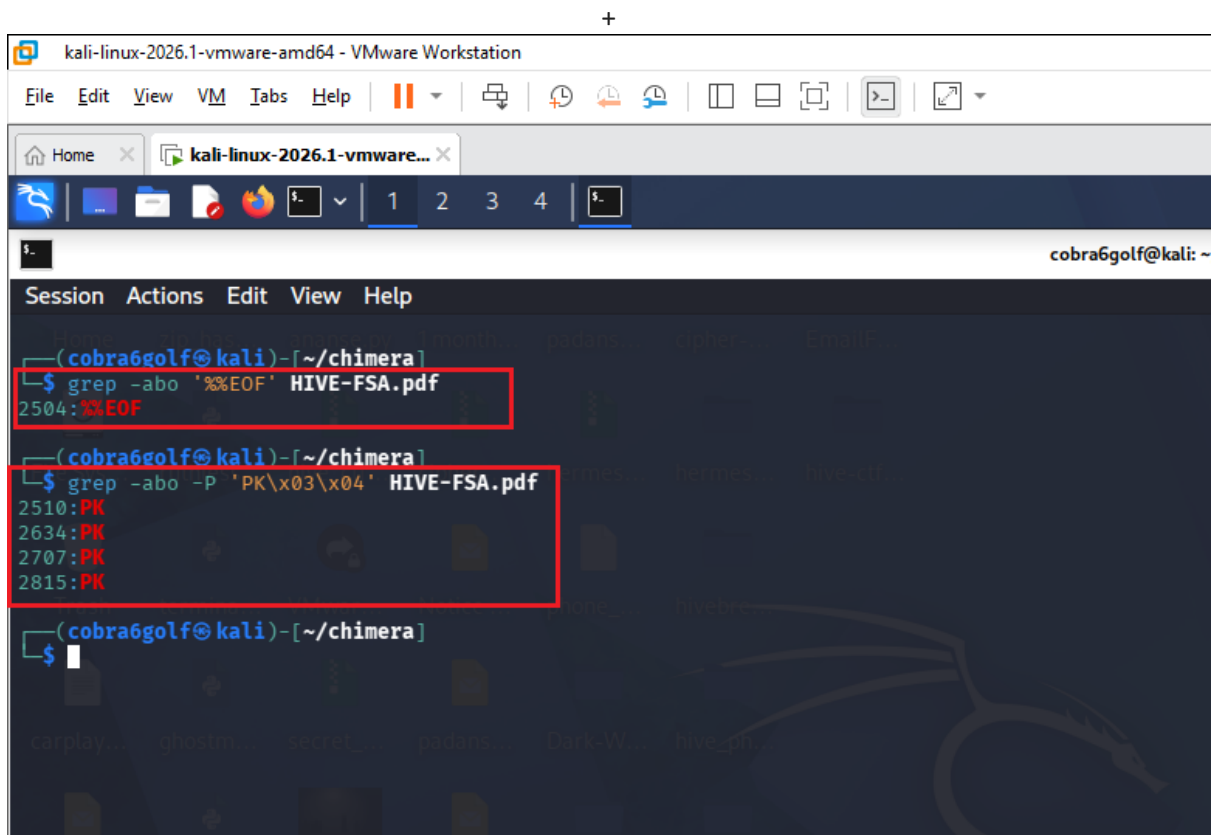


Figure 3. Offset of the %%EOF marker and the four ZIP signatures found after it.

Box	What it shows	Why it matters
Red	grep -abo '%%EOF' returns one hit at offset 2504.	This is where the real PDF ended
Red	grep -abo -P 'PK\x03\x04' returns hits at 2510, 2634, 2707 and 2815.	Four ZIP entry headers started just after the end of the PDF. Hidden files were stored there.

So, the PDF itself ends at byte 2,504 and about 190 kB of extra data follows it (192,947 minus 2,510 is 190,437 bytes). The four signatures match the four files I later found in the archive.

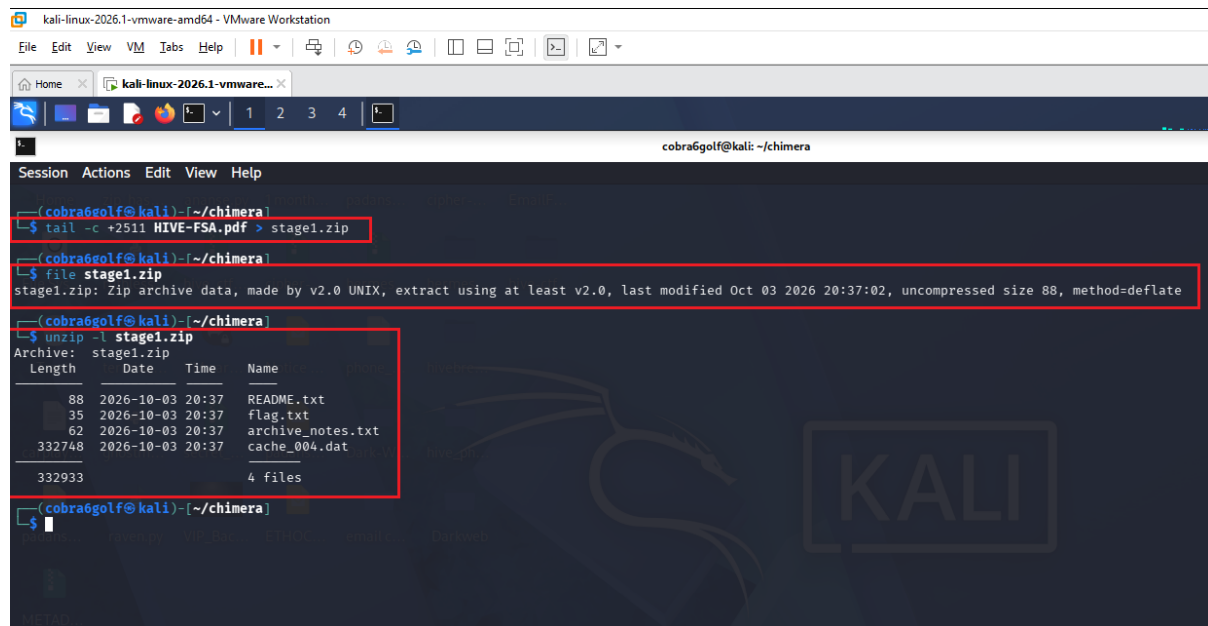
Result: Hidden ZIP data sits after the end of the PDF, starting at byte offset 2510.

5.3 Carving the first archive

I cut everything from offset 2510 to the end of the file into a new file. tail counts from 1, so I used tail -c +2511 (Figure 4):

```
tail -c +2511 HIVE-FSA.pdf > stage1.zip
```

The file command confirmed a ZIP archive and unzip -l listed four files.



```

kali-linux-2026.1-vmware-amd64 - VMware Workstation
File Edit View VM Tabs Help
kali-linux-2026.1-vmware-amd64
cobra6golf@kali: ~/chimera
Session Actions Edit View Help
(cobra6golf@kali)~/chimera
$ tail -c +2511 HIVE-FSA.pdf > stage1.zip
(cobra6golf@kali)~/chimera
$ file stage1.zip
stage1.zip: Zip archive data, made by v2.0 UNIX, extract using at least v2.0, last modified Oct 03 2026 20:37:02, uncompressed size 88, method=deflate
(cobra6golf@kali)~/chimera
$ unzip -l stage1.zip
Archive: stage1.zip
Length Date Time Name
  88  2026-10-03 20:37  README.txt
  35  2026-10-03 20:37  flag.txt
  62  2026-10-03 20:37  archive_notes.txt
332748 2026-10-03 20:37  cache_004.dat
332933
4 files
(cobra6golf@kali)~/chimera
$

```

Figure 4. First ZIP carved out of the PDF (stage1.zip) and its file listing.

Box	What it shows	Why it matters
Red	<code>tail -c +2511 HIVE-FSA.pdf > stage1.zip</code>	Copied everything from offset 2510 to the end of the file. <code>tail</code> counts from 1, so I added 1.
Red	<code>file stage1.zip</code> reported a ZIP archive, last modified Oct 03 2026 20:37:02.	Confirmed the carved data is a valid ZIP archive.
Red	<code>unzip -l stage1.zip</code> lists 4 files totaling 332,933 bytes.	Showed what was hidden. <code>cache_004.dat</code> was the largest item at 332,748 bytes.

File	Size (bytes)	Notes
README.txt	88	Short text note
flag.txt	35	Contained a flag-style string
archive_notes.txt	62	Short text note
cache_004.dat	332,748	Largest item. The real type was unknown at this point

5.4 Reading the first archive and the first decoy

I extracted the archive into `s1` and read the three text files (Figure 5). The README says the files are an internal recovery cache from Project Chimera. The notes file says to validate file contents before use. The file `flag.txt` held this:

```
HIVECTF{ARCHIVE_FOUND_CASE_CLOSED}
```

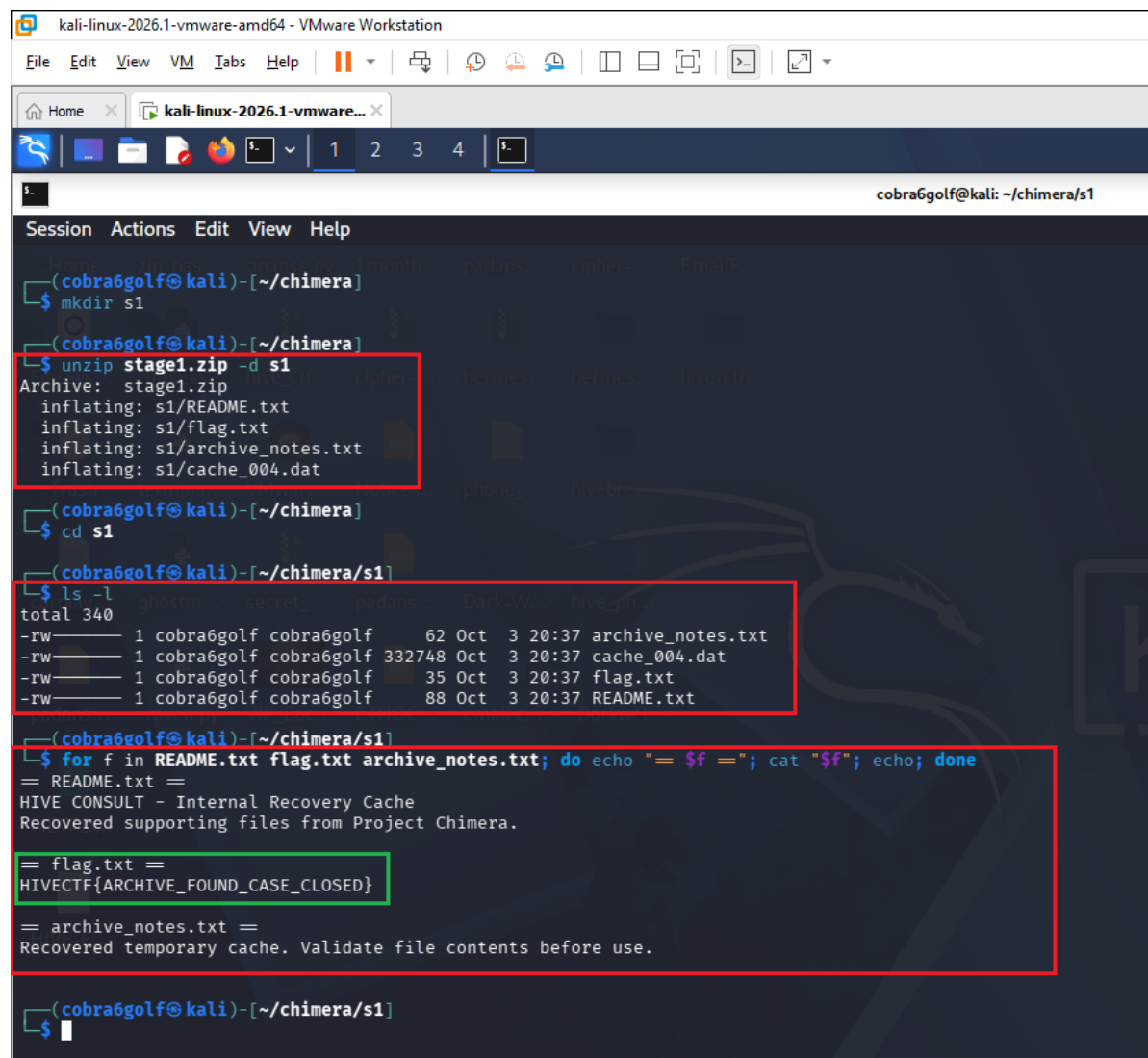


Figure 5. stage1.zip extracted to s1 and the three text files read, including the first decoy flag.

Box	What it shows	Why it matters
Red	unzip stage1.zip -d s1 extracts all four files.	The archive extracted cleanly into a separate folder
Red	ls -l shows sizes of 62, 332,748, 35 and 88 bytes.	Sizes matched the archive listing, so nothing was lost here
Red	Loop that prints README.txt, flag.txt and archive_notes.txt.	The README named Project Chimera. The notes said we needed to validate file contents before use
Green	HIVECTF{ARCHIVE_FOUND_CASE_CLOSED} in flag.txt.	First flag-style string. I classed it as Decoy 1 (see Section 6).

I did not accept this one. It was a loose text file that said the case was closed, while a 332 kB data file was still unexplained and the notes told me to validate before trusting anything. I recorded it as **Decoy 1**.

Result: One decoy recorded. cache_004.dat still needed to be identified.

5.5 Decoding cache_004.dat into an email

The file command described cache_004.dat as ASCII text with very long lines (65,536 characters) and no line breaks. That looked like encoded text, not a normal data file. The first 64 characters began with RnJvbTog, which is typical base64 (Figure 6). Decoding that first piece gave readable text: **From: digitalforensics@hiveconsult.com.**

I decoded the whole file into cache.eml. The file command then described it as mail text with CRLF line endings. The headers are below.

```

kali-linux-2026.1-vmware-amd64 - VMware Workstation
File Edit View VM Tabs Help
Home kali-linux-2026.1-vmware...
cobra6golf@kali: ~/chimera/s1
Session Actions Edit View Help
(cobra6golf@kali)~/chimera/s1
$ file cache_004.dat
cache_004.dat: ASCII text, with very long lines (65536), with no line terminators

(cobra6golf@kali)~/chimera/s1
$ head -c 64 cache_004.dat; echo
RnJvbTogZGlnaXRhbGZvcuVuc2lhc0BoaXZlY29uc3VsdC5jb20NClRvOiBleGVj

(cobra6golf@kali)~/chimera/s1
$ echo 'RnJvbTogZGlnaXRhbGZvcuVuc2lhc0BoaXZlY29uc3VsdC5jb20NClRvOiBleGVj' | base64 -d
From: digitalforensics@hiveconsult.com
To: exec

(cobra6golf@kali)~/chimera/s1
$ base64 -d cache_004.dat > cache.eml

(cobra6golf@kali)~/chimera/s1
$ file cache.eml
cache.eml: news or mail, ASCII text, with CRLF line terminators

(cobra6golf@kali)~/chimera/s1
$ head -15 cache.eml
From: digitalforensics@hiveconsult.com
To: executive.office@hiveconsult.com
Cc: incident.response@hiveconsult.com
Date: Sat, 03 Oct 2026 23:15:00 +0000
Subject: CRITICAL: Evidence Repository Manipulation Detected - Project Chimera
Message-ID: <HIVE-IR-CHIMERA-033@hiveconsult.com>
X-HIVE-Case-ID: CHIMERA-033
X-HIVE-Evidence-Class: RESTRICTED
X-HIVE-Flag-01: HIVECTF{RAW_EMAIL_HEADER_REVEALS_FLAG_ONE}
MIME-Version: 1.0
Content-Type: multipart/mixed; boundary="=====1032481858911691346=="

--=====1032481858911691346==
Content-Type: text/plain; charset="utf-8"
Content-Transfer-Encoding: quoted-printable

```

Figure 6. cache_004.dat identified as base64, decoded to cache.eml, and the email headers showing Flag 1.

Box	What it shows	Why it matters
Red	file cache_004.dat: ASCII text, very long lines (65536), no line terminators.	Not a normal data file. It looked like an encoded text
Red	head -c 64 returns RnJvbTog...	The pattern was typical of base64
Red	Decoding that sample prints From: digitalforensics@hiveconsult.com and To: exec.	Proves the text is a base64 encoded email

Box	What it shows	Why it matters
Red	<code>base64 -d cache_004.dat > cache.eml</code> , then file <code>cache.eml</code> .	The full decode was a mail file with CRLF line endings
Red	<code>head -15 cache.eml</code> shows the headers and the start of the MIME body.	Gave us sender, recipients, date, subject and the custom X-HIVE headers
Green	X-HIVE-Flag-01: <code>HIVECTF{RAW_EMAIL_HEADER_REVEALS_FLAG_ONE}</code>	Flag 1, found in the raw headers.

Header	Value
From	digitalforensics@hiveconsult.com
To	executive.office@hiveconsult.com
Cc	incident.response@hiveconsult.com
Date	Sat, 03 Oct 2026 23:15:00 +0000
Subject	CRITICAL: Evidence Repository Manipulation Detected - Project Chimera
Message-ID	<HIVE-IR-CHIMERA-033@hiveconsult.com>
X-HIVE-Case-ID	CHIMERA-033
X-HIVE-Evidence-Class	RESTRICTED
X-HIVE-Flag-01	<code>HIVECTF{RAW_EMAIL_HEADER_REVEALS_FLAG_ONE}</code>
Content-Type	multipart/mixed

The X-HIVE-Flag-01 header is a custom field that a mail client would not normally show. It names itself as flag one and sits in the raw headers of the decoded email.

Result: *Flag 1 recovered from the raw email headers.*

5.6 Finding the email attachment

The email was multipart, so I searched it for MIME headers and boundary lines (Figure 7). The message has a plain text body (quoted-printable) and one attachment, base64 encoded, named `meeting_evidence.dat`. The closing boundary was at line 3225, so most of the email is the attachment.

```

(cobra6golf@kali) - [~/chimera/s1]
$ grep -a -n -E '^([Content-Type|Content-Disposition|Content-Transfer-Encoding])' cache.eml
11:Content-Type: multipart/mixed; boundary="1032481858911691346="
13:--1032481858911691346=
14:Content-Type: text/plain; charset="utf-8"
15:Content-Transfer-Encoding: quoted-printable
43:--1032481858911691346=
44:Content-Type: application/octet-stream
45:Content-Transfer-Encoding: base64
46:Content-Disposition: attachment; filename="meeting_evidence.dat"
3225:--1032481858911691346=
(cobra6golf@kali) - [~/chimera/s1]
$

```

Figure 7. MIME structure of cache.eml showing the attachment named meeting_evidence.dat.

Box	What it shows	Why it matters
Red	grep for Content-Type, Content-Disposition, Content-Transfer-Encoding and boundary lines, with line numbers.	Mapped the structure of the email without reading all 3,225 lines
Green	Lines 11 and 13: multipart/mixed and the first boundary.	The email had more than one part
Green	Lines 43 to 46: application/octet-stream, base64, attachment named meeting_evidence.dat.	The second part was a base64 attachment. The closing boundary at line 3225 showed it makes up most of the email

I used Python's email library to pull out the parts and save the attachment (Figure 8). It listed three parts which is the multipart container, the text body, and the attachment. The saved file was 180,924 bytes.

```

kali-linux-2026.1-vmware-amd64 - VMware Workstation
File Edit View VM Tabs Help
kali-linux-2026.1-vmware...
cobra6golf@kali: ~/chimera/s1
Session Actions Edit View Help
cobra6golf@kali:~/chimera/s1
$ python3 -<< EOF
import email
from email import policy
m = email.message_from_bytes(open('cache.eml', 'rb').read(), policy=policy.default)
for i, p in enumerate(m.walk()):
    print(i, p.get_content_type(), p.get_filename())
    if p.get_filename():
        open(p.get_filename(), 'wb').write(p.get_payload(decode=True))
        print("saved -> ", p.get_filename())
EOF
ls -l meeting_evidence.dat
file meeting_evidence.dat
sha256sum meeting_evidence.dat
0 multipart/mixed None
1 text/plain None
2 application/octet-stream meeting_evidence.dat
saved -> meeting_evidence.dat
-rw-rw-r-- 1 cobra6golf cobra6golf 180924 Oct 4 03:43 meeting_evidence.dat
meeting_evidence.dat: JPEG image data, JFIF standard 1.01, aspect ratio, density 1x1, segment length 16, Exif Standard: [TIFF image data, big-endian, direntries=4, description=HIVECTF{METADATA_WHIS
PERS THE FINAL FLAG}] software=HIVE CONSULT CTF Generator), baseline, precision 8, 1500x930, components 3
de20a8e74e8186797d2be05707e7400a013f74ce6a3167b0cbafdfde9493c88b meeting_evidence.dat
cobra6golf@kali:~/chimera/s1

```

Figure 8. Attachment extracted with Python and identified as a JPEG image.

Box	What it shows	Why it matters
Red	Python script that walks the email parts and writes the attachment to disk, followed by <code>ls -l</code> , file and sha256sum.	Extracted the attachment with a proper MIME parser. Parts found were multipart/mixed, text/plain, application/octet-stream
Green	file output: JPEG image data, JFIF 1.01, 1500x930, with an EXIF description field.	The .dat file was really a JPEG. The description field already showed the metadata flag text
Red	<code>ls -l</code> shows 180,924 bytes. SHA-256 begins de20a8e7.	Size and hash recorded for this file

Property	Value
Name in the email	meeting_evidence.dat
Declared type	application/octet-stream
Real type (from file)	JPEG image data, JFIF 1.01, 1500 x 930, 3 components
Size	180,924 bytes
SHA-256	de20a8e74e8186797d2be05707e7400a013f74ce6a3167b0cbafdfde9493c88b

The .dat name did not match the content. The file begins with JPEG data, so I copied it to `meeting.jpg` and opened it (Figure 9). It displayed a HIVE CONSULT page that reads "You restored the evidence as a JPEG" and shows this flag:

HIVECTF{FILE_SIGNATURE_RESTORES_JPEG_FLAG_TWO}

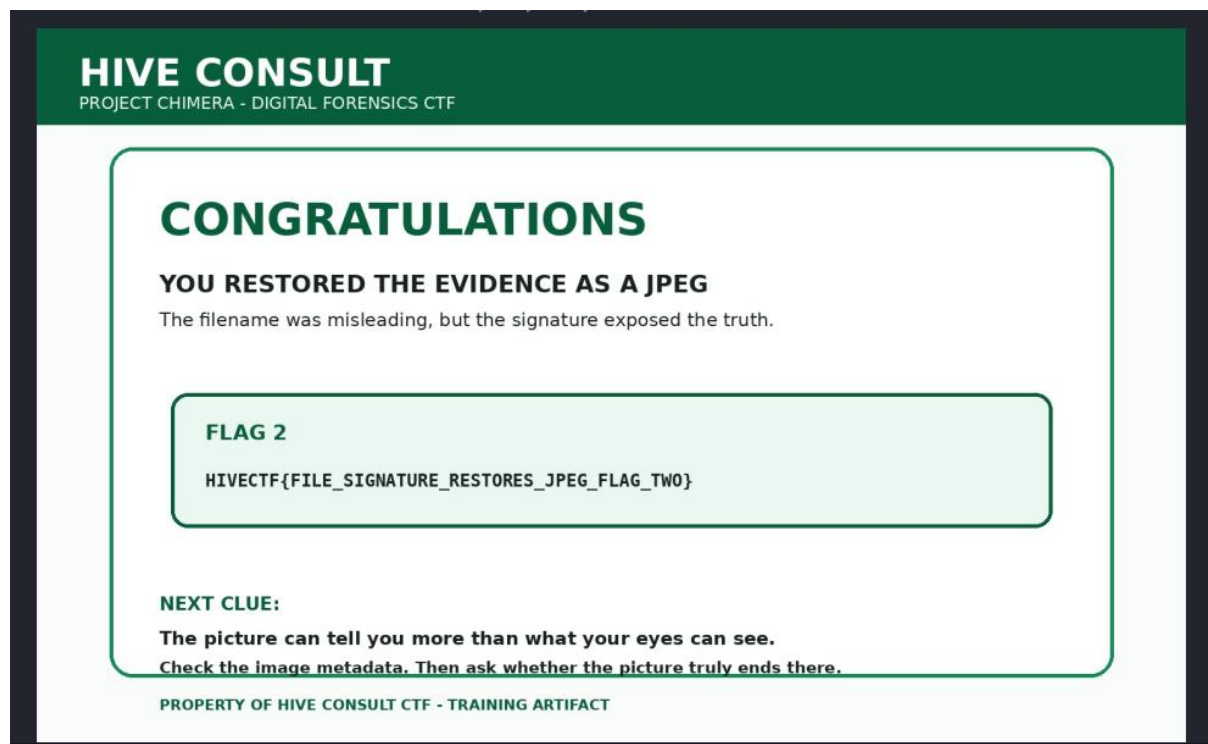


Figure 9. The JPEG opened as an image, showing Flag 2 and the next clue.

Box	What it shows	Why it matters
Image	HIVE CONSULT banner, "Project Chimera - Digital Forensics CTF".	The file renders as a real image
Image	"You restored the evidence as a JPEG. The filename was misleading, but the signature exposed the truth."	Confirmed the .dat name was wrong
Image	Flag 2: <code>HIVECTF{FILE_SIGNATURE_RESTORES_JPEG_FLAG_TWO}</code>	Flag 2
Image	Next was to check the image metadata, then ask whether the picture truly ends there.	Pointed to Sections 5.7 and 5.8

The page also gave a clue for us to check the image metadata, then ask whether the picture truly ends there.

Result: Flag 2 recovered by identifying the attachment by its signature, not its name.

5.7 Reading the image metadata

Following the clue, I ran `exiftool -a -u -G1 meeting.jpg` (Figure 10).

```

kali-linux-2026.1-vmware-amd64 - VMware Workstation
File Edit View VM Tabs Help
Home x kali-linux-2026.1-vmware... x
cobra6golf@kali: ~/chimera/s1
Session Actions Edit View Help
(cobra6golf@kali)-[~/chimera/s1]
$ cp meeting_evidence.dat meeting.jpg
(cobra6golf@kali)-[~/chimera/s1]
$ xdg-open meeting.jpg
(cobra6golf@kali)-[~/chimera/s1]
$ exiftool -a -u -G1 meeting.jpg
[ExifTool]      ExifTool Version Number      : 13.50
[System]        File Name                    : meeting.jpg
[System]        Directory                    : .
[System]        File Size                    : 181 kB
[System]        File Modification Date/Time   : 2026:10:04 03:59:50-04:00
[System]        File Access Date/Time        : 2026:10:04 04:00:09-04:00
[System]        File Inode Change Date/Time   : 2026:10:04 03:59:50-04:00
[System]        File Permissions              : -rw-rw-r--
[File]          File Type                    : JPEG
[File]          File Type Extension          : jpg
[File]          MIME Type                    : image/jpeg
[File]          Exif Byte Order               : Big-endian (Motorola, MM)
[File]          Image Width                  : 1500
[File]          Image Height                 : 930
[File]          Encoding Process              : Baseline DCT, Huffman coding
[File]          Bits Per Sample              : 8
[File]          Color Components              : 3
[File]          Y Cb Cr Sub Sampling          : YCbCr4:2:0 (2 2)
[JFIF]          JFIF Version                 : 1.01
[JFIF]          Resolution Unit               : None
[JFIF]          X Resolution                  : 1
[JFIF]          Y Resolution                  : 1
[IFD0]          Image Description             : HIVECTF{METADATA_WHISPERS_THE_FINAL_FLAG}
[IFD0]          Software                     : HIVE CONSULT CTF Generator
[IFD0]          Artist                       : HIVE CONSULT Digital Forensics Team
[IFD0]          XP Comment                    : 桔 浩条 溪徽戡晚拈 桴 痲摩溪散掬戢
[Composite]    Image Size                    : 1500x930
[Composite]    Megapixels                    : 1.4
(cobra6golf@kali)-[~/chimera/s1]
$

```

Figure 10. ExifTool metadata for meeting.jpg, including the Image Description flag.

Box	What it shows	Why it matters
Red	cp meeting_evidence.dat meeting.jpg and xdg-open meeting.jpg.	Copied with the correct extension, then opened as an image
Red	Image Description: <i>HIVECTF{METADATA_WHISPERS_THE_FINAL_FLAG}</i>	Flag 3, was stored in the EXIF data
Green	XP Comment showed unreadable characters.	A field worth a looking into to see the note below this tab

Field	Value
Image Description	<i>HIVECTF{METADATA_WHISPERS_THE_FINAL_FLAG}</i>
Software	HIVE CONSULT CTF Generator
Artist	HIVE CONSULT Digital Forensics Team
XP Comment	Displayed as unreadable characters. Likely the same hint text, shown with the wrong encoding (see below).

Field	Value
Image size	1500 x 930 pixels (1.4 megapixels)
Exif byte order	Big-endian (Motorola)

The Image Description field holds a flag. The word "FINAL" in its text is misleading, because I had not yet found the end of the image data. I treated that word as bait and kept going. I assigned this one as **Flag 3** because the previous page sent me to the metadata, and this is the flag the metadata held. My reasoning is in Section 6.

Note on the XP Comment field. ExifTool reads this field as UTF-16, two bytes per character. The first characters on screen (桔, 浩, 条) split into the byte pairs Th, im and ag when read the other way round. That matches the start of the sentence "The image ends before the evidence does." those strings found in the same file. So, the field most likely holds that sentence, stored as plain text. I did not run a raw-byte decode to confirm it.

Result: Flag 3 recovered from the EXIF Image Description.

5.8 Finding data after the end of the JPEG

I ran strings on the image (Figure 11). Near the top, besides the flag and the author text, was the line "The image ends before the evidence does." Near the bottom were the names README_FINAL.txt, FLAG4.txt and final_evidence.dat, followed by PK. Those are ZIP file names stored inside the image file.

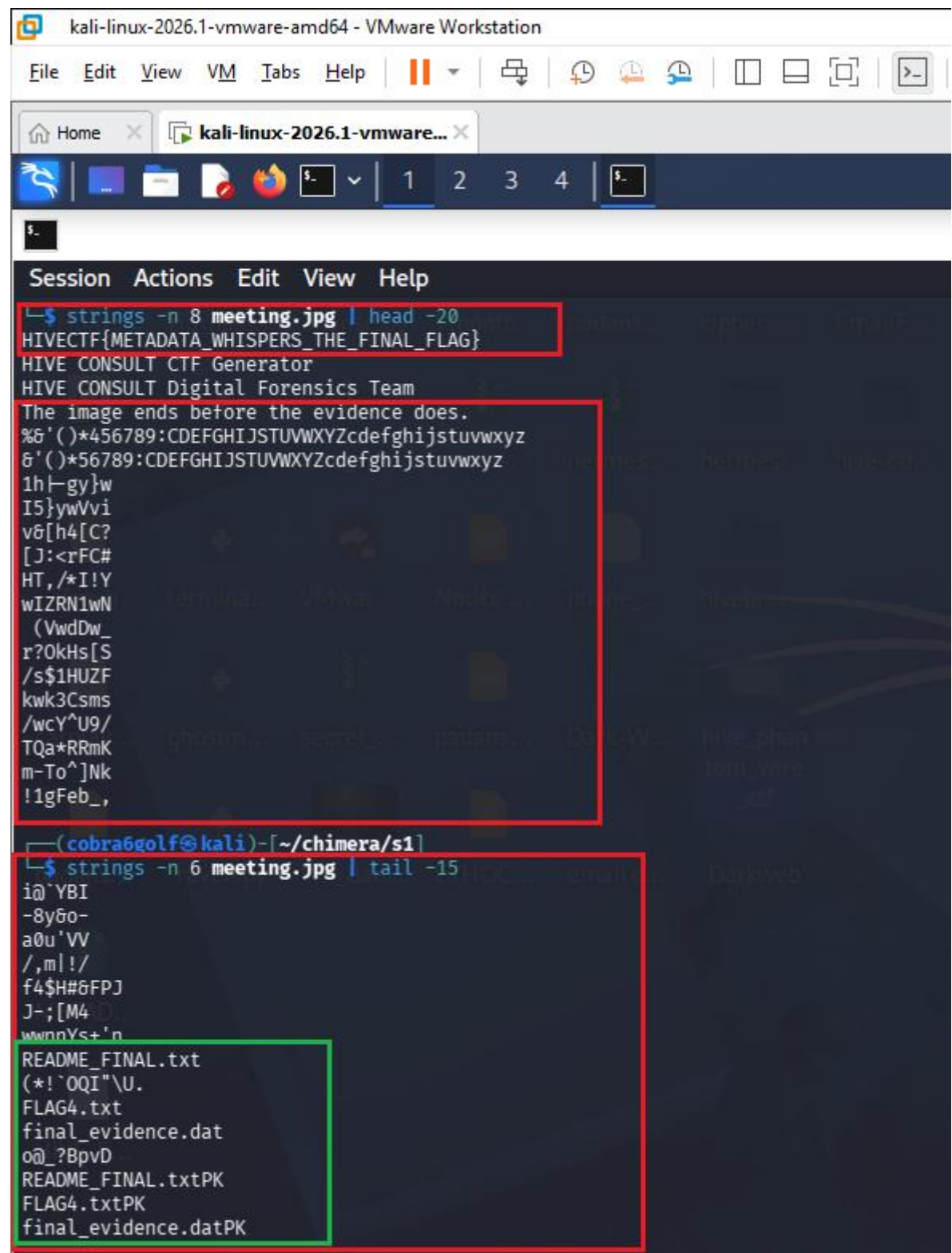


Figure 11. Strings from meeting.jpg. The tail shows file names stored after the image.

Box	What it shows	Why it matters
Red	strings -n 8 meeting.jpg head -20: the flag, the software and artist names, and the line "The image ends before the evidence does."	Readable text that were stored in the file. The sentence is a hint that more data follows the image

Box	What it shows	Why it matters
Red	Rest of the output is random-looking characters.	This was a compressed picture data, not meaningful text
Red	<code>strings -n 6 meeting.jpg tail -15</code>	Showed us the last readable text in the file
Green	README_FINAL.txt, FLAG4.txt, final_evidence.dat, each followed by PK.	File names of a ZIP archive that were stored at the end of the image

A JPEG ends with the marker FF D9. I wrote a short Python script that finds the last FF D9 in the file and reports what follows it (Figure 12).

The screenshot shows a Kali Linux terminal window with the following content:

```

(cobra6golf@kali)-[~/chimera/s1]
$ python3 - << EOF
d = open('meeting.jpg', 'rb').read()
i = d.rfind(b'\xff\xd9') + 2
print("total file size :", len(d))
print("image really ends at byte:", i)
print("bytes after the end:", len(d) - i)
print("first 8 bytes after end:", d[i:i+8])
EOF
total file size : 180924
image really ends at byte: 179021
bytes after the end: 1903
first 8 bytes after end: b'PK\x03\x04\x14\x00\x00\x00'

```

Figure 12. Calculation of where the JPEG really ends and how many bytes follow.

Box	What it shows	Why it matters
Red	Python script: reads the file, finds the last FF D9 (end of JPEG), adds 2, then prints sizes and the next 8 bytes.	Worked out exactly where the image stops and what followed
Green	Total 180,924 bytes. Image ends at byte 179,021. 1,903 bytes after. Next bytes: PK\x03\x04...	1,903 extra bytes sat after the image and they began with a ZIP signature.

Measurement	Value
Total file size	180,924 bytes
JPEG data ends at byte	179,021
Bytes after the end	1,903
First 8 bytes after the end	50 4B 03 04 14 00 00 00 (the ZIP signature)

A ZIP signature sitting exactly at the end of the image data confirmed that the boundary was correct. I carved the trailing bytes two ways, with `tail -c +179022 meeting.jpg` and with `dd if=meeting.jpg of=stage2.zip bs=1 skip=179021` (Figure 13). The resulting stage2.zip was 1,903 bytes, the same as my calculation, and `unzip -l` listed three files.

```

kali-linux-2026.1-vmware-amd64 - VMware Workstation
File Edit View VM Tabs Help
kali-linux-2026.1-vmware-amd64
cobra6golf@kali: ~/chimera/s1
Session Actions Edit View Help
cobra6golf@kali)~/.chimera/s1
$ tail -c +179022 meeting.jpg > stage2.zip
cobra6golf@kali)~/.chimera/s1
$ dd if=meeting.jpg of=stage2.zip bs=1 skip=179021
1903+0 records in
1903+0 records out
1903 bytes (1.9 kB, 1.9 KiB) copied, 0.00249508 s, 763 kB/s
cobra6golf@kali)~/.chimera/s1
$ ls -l stage2.zip
file stage2.zip
unzip -l stage2.zip
-rw-rw-r-- 1 cobra6golf cobra6golf 1903 Oct  4 04:19 stage2.zip
stage2.zip: Zip archive data, made by v2.0 UNIX, extract using at least v2.0, last modified Oct 03 2026 20:37:02, uncompressed size 149, method=deflate
Archive: stage2.zip
Length      Date       Time       Name
  149    2026-10-03  20:37   README_FINAL.txt
   36    2026-10-03  20:37   FLAG4.txt
 2355    2026-10-03  20:37   final_evidence.dat
  2540                   3 files
cobra6golf@kali)~/.chimera/s1
$

```

Figure 13. Second ZIP carved from the end of the JPEG and its file listing.

Box	What it shows	Why it matters
Red	<code>tail -c +179022 meeting.jpg > stage2.zip</code>	Carved the trailing data. <code>tail</code> counts from 1, so I added 1 to 179,021 as usual
Red	<code>dd if=meeting.jpg of=stage2.zip bs=1 skip=179021</code> copies 1,903 bytes.	Second method for the same job. The byte count matched my calculation made
Red	<code>ls -l</code> , file and <code>unzip -l</code> on stage2.zip.	1,903 bytes, a valid ZIP archive that was last modified on Oct 03 2026 20:37:02
Green	Listing: README_FINAL.txt (149), FLAG4.txt (36), final_evidence.dat (2,355), total 2,540 bytes.	Contents of the second hidden archive

File	Size (bytes)	Notes
README_FINAL.txt	149	Text note
FLAG4.txt	36	Contained a flag-style string
final_evidence.dat	2,355	Real type was unknown at this point

5.9 Reading the second archive and the second decoy

I extracted the archive into s2 (Figure 14). The README said a readable image may still contain bytes after its legitimate end, and that every recovered object must be validated by signature. The file FLAG4.txt held this:

```
HIVECTF{FINAL_FLAG_FOUND_TOO_EARLY}
```

```

kali-linux-2026.1-vmware-amd64 - VMware Workstation
File Edit View VM Tabs Help
kali-linux-2026.1-vmware...
$ _
Session Actions Edit View Help
(cobra6golf@kali)-[~/chimera/s1]
$ mkdir s2
unzip stage2.zip -d s2
cd s2
ls -l
cat README_FINAL.txt
echo
cat FLAG4.txt
echo
file final_evidence.dat
head -c 8 final_evidence.dat | od -An -tx1
Archive: stage2.zip
  inflating: s2/README_FINAL.txt
  inflating: s2/FLAG4.txt
  inflating: s2/final_evidence.dat
total 12
-rw-r--r-- 1 cobra6golf cobra6golf 2355 Oct  3 20:37 final_evidence.dat
-rw-r--r-- 1 cobra6golf cobra6golf   36 Oct  3 20:37 FLAG4.txt
-rw-r--r-- 1 cobra6golf cobra6golf  149 Oct  3 20:37 README_FINAL.txt
HIVE CONSULT - FINAL EVIDENCE CACHE
A readable image may still contain bytes after its legitimate end.
Validate every recovered object by signature.
HIVECTF{FINAL_FLAG_FOUND_TOO_EARLY}
final_evidence.dat: PDF document, version 1.4, 1 page(s)
25 50 44 46 2d 31 2e 34
(cobra6golf@kali)-[~/chimera/s1/s2]
$ 

```

Figure 14. stage2.zip extracted; README_FINAL.txt, the second decoy flag, and the PDF signature check.

Box	What it shows	Why it matters
Red	mkdir s2, unzip, ls -l, cat on the two text files, file and head -c 8 od on the .dat file.	Extracted the archive and checked each item
Red	Output of README_FINAL.txt: a readable image may still contain bytes after its legitimate end, and every object must be validated by signature.	Tells me to check file types instead of trusting names or flag files
Green	FLAG4.txt: HIVECTF{FINAL_FLAG_FOUND_TOO_EARLY}	Decoy 2
Green	file says PDF document, version 1.4. First bytes 25 50 44 46 2d 31 2e 34.	Those bytes spell %PDF-1.4, so final_evidence.dat was a PDF.

The text itself says it was found too early, and the README had just told me to validate by signature. I did not accept it and recorded it as **Decoy 2**.

5.10 Validating the last file

The file command described `final_evidence.dat` as a PDF document, version 1.4. I checked the first bytes with `od`, and they were `25 50 44 46 2d 31 2e 34`, which spell `%PDF-1.4`. I copied it to `final.pdf` and read it (Figure 15).

`pdftotext` showed a page titled "Project Chimera - Final Evidence Recovery". The page stated that the examiner identified data beyond the legitimate end of the JPEG, carved the appended archive, and validated the true file type. It holds this flag:

HIVECTF{JPEG_EOI_WAS_NOT_THE_END}

The screenshot shows a Kali Linux terminal window with the following content:

```

kali-linux-2026.1-vmware-amd64 - VMware Workstation
File Edit View VM Tabs Help
Home kali-linux-2026.1-vmware...
$ _
Session Actions Edit View Help
(cobra6golf@kali) - [~/chimera/s1/s2]
$ cp final_evidence.dat final.pdf
pdftotext final.pdf -
pdftinfo final.pdf
grep -a -o 'HIVECTF{[^}]*}' final.pdf
HIVE CONSULT
PROJECT CHIMERA - FINAL EVIDENCE RECOVERY

FINAL FLAG RECOVERED
The examiner correctly identified data beyond the legitimate end of the JPEG image,
carved the appended archive, and validated the true file type of the final evidence object.

This confirms that a readable image can still contain additional concealed evidence.

FLAG 4
HIVECTF{JPEG_EOI_WAS_NOT_THE_END}

PROPERTY OF HIVE CONSULT CTF - RESTRICTED TRAINING ARTIFACT

Title:          untitled
Subject:        unspecified
Keywords:
Author:         anonymous
Creator:        anonymous
Producer:       ReportLab PDF Library - (opensource)
CreationDate:   Sat Oct 3 16:37:02 2026 EDT
ModDate:       Sat Oct 3 16:37:02 2026 EDT
Custom Metadata: no
Metadata Stream: no
Tagged:        no
UserProperties: no
Suspects:      no
Form:          none
JavaScript:     no
Pages:         1
Encrypted:      no
Page size:     595.276 x 841.89 pts (A4)
Page rot:      0
File size:     2355 bytes
Optimized:     no
PDF version:   1.4

```

Figure 15. final.pdf read with pdftotext and pdftinfo, showing Flag 4.

Box	What it shows	Why it matters
Red	cp final_evidence.dat final.pdf, pdftotext, pdftinfo and grep -a -o 'HIVECTF{[^}]*}'.	Renamed the file, read its text and details and pulls out any flag inside
Green	HIVECTF{JPEG_EOI_WAS_NOT_THE_END}	Flag 4. The page text said the examiner found data beyond the legitimate end of the JPEG.

PDF property	Value
Size	2,355 bytes
Pages / page size	1 page, A4 (595.276 x 841.89 pts)
Producer	ReportLab PDF Library (opensource)
Created / modified	Sat Oct 3 16:37:02 2026 EDT
Encrypted / JavaScript / forms	No / No / None
Custom metadata	None

Result: Flag 4 recovered. The file was a plain PDF with no scripts, forms or encryption.

6. Flag Assessment

The brief said there were four genuine flags and two decoys. I found six flag-style strings, which matched that count. The table shows how I classified each one.

No.	Flag	Where I found it	Verdict	Reason
1	HIVECTF{RAW_EMAIL_HEADER_REVEALS_FLAG_ONE}	X-HIVE-Flag-01 header in cache.eml	Genuine	Labelled as flag one, inside the decoded email.
2	HIVECTF{FILE_SIGNATURE_RESTORES_JPEG_FLAG_TWO}	Picture content of meeting.jpg	Genuine	Labelled Flag 2 on the image, reached by fixing the file type.
3	HIVECTF{METADATA_WHISPERS_THE_FINAL_FLAG}	EXIF Image Description of meeting.jpg	Genuine	The image clue pointed to metadata. The word "final" is misleading.
4	HIVECTF{JPEG_EOI_WAS_NOT_THE_END}	Text of final.pdf	Genuine	Labelled Flag 4 in a validated PDF at the end of the chain.
D1	HIVECTF{ARCHIVE_FOUND_CASE_CLOSED}	flag.txt in the first ZIP	Decoy	Loose text file that claims the case is closed while evidence remained.
D2	HIVECTF{FINAL_FLAG_FOUND_TOO_EARLY}	FLAG4.txt in the second ZIP	Decoy	Named like Flag 4, but the real Flag 4 was inside a different file.

Flags 1, 2 and 4 carry their own numbers in the evidence. Flag 3 does not, so I numbered it by position in the chain. If the case author numbers it differently, the set of four genuine flags stays the same.

7. Chain of Recovered Files

The table follows each file from the original exhibit to the final result.

#	Item	Where it came from	Real type	Outcome
1	HIVE-FSA.pdf	Exhibit	PDF	Extra data after %%EOF at byte 2504
2	stage1.zip	Byte 2510 of the PDF	ZIP, 4 files	Decoy 1 and cache_004.dat
3	cache_004.dat	Inside stage1.zip	Base64 text	Decoded to an email
4	cache.eml	Decoded from item 3	Email (MIME)	Flag 1 in headers

#	Item	Where it came from	Real type	Outcome
5	meeting_evidence.dat	Attachment in item 4	JPEG, 1500 x 930	Flag 2 (picture), Flag 3 (metadata)
6	stage2.zip	Byte 179021 of the JPEG	ZIP, 3 files	Decoy 2 and final_evidence.dat
7	final_evidence.dat	Inside stage2.zip	PDF, 1 page	Flag 4

8. Conclusion

The board audit PDF is a valid, readable document, but it carries a chain of hidden material behind it. From the original file I recovered a ZIP archive, an encoded email, a disguised JPEG, a second ZIP archive and a final PDF. I recovered all four genuine flags and identified both decoys.

Three points stand out from this work:

- A file can pass as legitimate and still hold more data. The PDF opened normally, and so did the JPEG.
- File names and extensions cannot be trusted. Two files called .dat were a JPEG and a PDF, and I only knew that from their signatures.
- Flag-style text that appears early, or that claims the work is finished, deserves suspicion. Both decoys did exactly that.

9. Limitations

- I recorded SHA-256 hashes for the working copy of the PDF and for the extracted JPEG only. I did not hash the other recovered files, and I did not hash the original PDF on the Desktop to compare it with my copy.
- The XP Comment field in the JPEG metadata displayed as unreadable characters. My reading that it holds the same hint sentence is based on the first few characters and the matching strings output, not on a full raw-byte decode.
- I viewed the image on screen but did not run any steganography tools on its pixel data.
- Flag 3 is numbered by its position in the chain, as explained in Section 6.
- The Invalid xref table warning was noted but not investigated further.

10. Recommendations

- Compare each file's size against where its structure ends. A large gap after %%EOF or FF D9 points to appended data.
- Identify every recovered object by its signature before opening it or trusting its name.
- Record a hash for each file at the moment it is recovered, so the chain of custody covers every layer.

- Dump the raw bytes of the XP Comment field and read them as plain text to confirm what it holds.
- Treat any file named like a flag or a final result as unverified until the surrounding evidence supports it.